LANDMARK METROPOLITAN UNIVERSITY INSTITUTE
THE PRIDE OF AFRICA

Object Oriented Programming (OOP)

SE309 – Python Programming with Project

Precious Oben, 2025/2026

INTRODUCTION

Object Oriented Programming is a programming paradigm that organizes code using **classes** and **objects**.

It models real world entities and their behaviors or interactions.

A class is a blueprint for creating objects (an object constructor).

An object is a an instance of a class.

To define a class, use the **class** keyword:

```
class NewClass:
  a = 7
```

Creating an object named object1:

```
Object1 = NewClass()
print(object1.a)
```

Attributes and Methods

Attributes in the real word are things that describe an entity. The same goes for classes and objects. Attributes are properties that describe objects and classes.

They are defined as parameters of a function:

```
class Level300Students:
    def __init__(self, name, age):
        self.name = name  # Instance attribute
        self.age = age

# Creating objects
student1 = Level300Students("Carl", 19)  # Fixed quotation marks
student2 = Level300Students("Grace", 20)  # Fixed quotation marks

print(student2.age)  # Prints the age of student2
```

Attributes and Methods

All classes have the **_init_** function which is executed when a class is initiated.
It is used to assign values to properties of all objects of a class and run all other necessary operations.

There are two main types of attributes when dealing with classes.

1. **Instance Attributes**

- They are unique to each object
- Defined using **self** inside the **_init_** method.

2. **Class Attributes**

- They are shared across all instances of a class.

A method in a class is a function that is associated with an object. It defines the behavior and actions that objects of the class can perform
Methods are important in ensuring modularity and reusability.
They are defined inside a class and can access the attributes (variables) of the class.

Attributes and Methods

1. **Instance Methods**

- Operate on Instance attributes.
- Require **self** as their first parameter.

2. **Class Methods**

- Operate on class attributes.
- Use the **@classmethod** decorator.

3. **Static Methods**

- Independent of class and instance attributes.
- Use the **@staticmethod** decorator.

Example

```
class Employee:
    company = "TechCorp"  # Class attribute

    def __init__(self, name, age):
        self.name = name  # Instance attribute
        self.age = age

    def show_details(self):  # Instance method
        return f"{self.name}, {self.age} years old, works at {self.company}"

    @classmethod
    def change_company(cls, new_company):  # Class method
        cls.company = new_company
```

Encapsulation, Inheritance and Polymorphism

Encapsulation

It refers to the concept of bundling data and the methods that operate on that data into a single unit. This approach restricts direct access to variables and methods, helping to prevent unintentional modifications to the data. To ensure controlled changes, an object's variables can only be modified through its methods. Such variables are referred to as private variables.

Private attributes are used with **"_"** or **"__".**

A class is a prime example of encapsulation because it encapsulates all its data, including member functions, variables, and other elements, within a single unit. This ensures controlled access and manipulation of the data through defined methods, promoting data security and modularity.

Example

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.__balance = balance  # Private attribute

    def get_balance(self):
        return self.__balance

    def deposit(self, amount):
        self.__balance += amount
```

Encapsulation, Inheritance and Polymorphism

Inheritance

It enables a class to inherit attributes from another class.

The class whose attributes are inherited is called the **Parent or Base Class .**

The class that inherits attributes is called the **Child** or **Derived** Class.

```
class Person:
  def __init__(self, fname, lname):
    self.firstname = fname
    self.lastname = lname

  def printname(self):
    print(self.firstname, self.lastname)
```

Encapsulation, Inheritance and Polymorphism

```
class Student(Person):
  pass


x = Student("Mike", "Olsen")
x.printname()
```

Encapsulation, Inheritance and Polymorphism

Polymorphism

It allows methods to act differently based on the objects that call them.

```
class Bird:
    def fly(self):
        return "Bird can fly"

class Penguin(Bird):
    def fly(self):
        return "Penguins can't fly"
```